Report: Programming Assignment - 1

Part 1A: Matrix multiplication kernel

1. Kernel Implementation: *matmul*

The matmul kernel executes the General Matrix Multiplication (GEMM) ***C = A * B*** on the GPU. Each thread is responsible for calculating exactly one element of the output matrix **C**.

- **Thread Mapping:** The kernel uses a 2D grid of thread blocks. We map the unique thread indices to matrix coordinates *(row, col)* as follows:
    - ***col_idx***: Corresponds to the matrix column (***N*** dimension), mapped to *blockIdx.x* and *threadIdx.x*.
    - ***row_idx***: Corresponds to the matrix row (***M*** dimension), mapped to *blockIdx.y* and *threadIdx.y*.
    - **Boundary Check:** A conditional check ***if (row_idx < M && col_idx < N)*** ensures that threads mapped outside the valid matrix dimensions do not perform calculations or access invalid memory.
- **Dot Product & Accumulation:** To compute the value for *C[row][col]*, the thread iterates through the shared dimension ***K*** (from *0* to *K-1*). Inside this loop, it computes the dot product of the mth row of A and the nth column of B.
- **Layout Handling (Row-Major vs. Column-Major):** The kernel supports flexible memory layouts using if/else logic based on the *layoutA* and *layoutB* flags. This is crucial because standard C++ uses Row-Major, while other systems (or CUBLAS) might use Column-Major.
    - **Row-Major:** Elements of a row are contiguous. The offset is calculated as ***row_index * width + column_index***.
    - **Column-Major:** Elements of a column are contiguous. The offset is calculated as ***column_index * height + row_index.***

2. Host Code & Memory Models

The host code is split into two functions to demonstrate different CUDA memory management techniques.

A. Explicit Memory (*run_explicit*)

This function demonstrates the traditional, manual approach to GPU memory management.

1. **Allocation:** Host memory is allocated using malloc, and device memory is allocated using ***cudaMalloc***.

Submitted by: Suhas Shastry Hebbe Sathish

2. **Data Movement:** We explicitly transfer input matrices from Host (CPU) to Device (GPU) using ***cudaMemcpy*** with the ***cudaMemcpyHostToDevice*** flag.
3. **Execution:** The kernel is launched, and CPU execution is synchronized using ***cudaEventSynchronize*** to measure precise timing.
4. **Retrieval:** Results are copied back to the host using ***cudaMemcpy*** (***cudaMemcpyDeviceToHost***) for verification.
5. **Cleanup:** Memory is freed explicitly using ***cudaFree*** and free.

B. Unified Memory (*run_unified*)

This function uses CUDA's Unified Virtual Addressing (UVA) to simplify memory management.

1. **Allocation:** We use ***cudaMallocManaged***, which allocates memory that is accessible by both the CPU and GPU. The CUDA driver handles the physical migration of data pages between host RAM and GPU VRAM automatically.
2. **No Explicit Copies:** We initialize data directly into these pointers on the CPU. We do not need ***cudaMemcpy***.
3. **Synchronization:** A critical call to ***cudaDeviceSynchronize***() is required after the kernel launch. This ensures the GPU has finished writing to the unified memory before the CPU attempts to read the results for verification.

3. Verification & Helpers

- **Grid Calculation:** The grid dimensions are calculated as (N + *block.x* - 1) / *block.x*. This standard integer division formula performs a "ceiling" operation, ensuring enough blocks are launched to cover the entire matrix even if dimensions are not multiples of the block size (16).
- **File I/O:** The ***readMatrixFile*** helper parses the provided text files, skipping the metadata headers (layout, rows, cols) to load the raw floating-point data correctly into memory for testing against the provided reference outputs.

Results for (16,16)

--- Explicit Memory (cudaMalloc) ---

Time: 0.105472 ms

--- Unified Memory (cudaMallocManaged) ---

Time: 0.38464 ms

Submitted by: Suhas Shastry Hebbe Sathish

Part 1B: GPGPU-Sim Performance Analysis

I ran the GPGPU-Sim simulation using the **sample_0** traces on two different block configurations (8,8) and (16,16).

1. Instructions Per Cycle (IPC)

- **Observed IPC:** My program achieved an IPC of **185.59** with the (8,8) block size and **178.21** with the (16,16) block size.
- **Calculation:** The simulator calculates IPC by taking the **total number of instructions executed** by all threads and dividing it by the **total number of GPU cycles** the simulation took to complete. Essentially, it measures the average throughput of valid instructions processed per clock tick.

2. Data Cache Miss Rates

- **Observed Rates:**
  - **L1 Cache:** The miss rate was **30.86%** for the (8,8) configuration, but it spiked to **56.92%** with the (16,16) blocks.
  - **L2 Cache:** The miss rate was significantly lower, at **5.88%** (8,8) and **11.11%** (16,16).
- **Calculation:** The simulator calculates these rates by tracking every memory request sent to the cache. The miss rate is simply the **total number of cache misses divided by the total number of cache accesses** (read/write requests). A "miss" is recorded whenever the requested data block is not found in the cache tag array, requiring a fetch from the next level of memory.

For 8,8

root@eb239500bcc6:~/gpgpu-sim_distribution/vectorAdd# grep -i "total_cache_miss_rate" output.txt && grep -i "gpu_ipc" output.txt

L1D_total_cache_miss_rate = 0.3086

L2_total_cache_miss_rate = 0.0588

gpu_ipc = 185.5878

For 16,16

root@eb239500bcc6:~/gpgpu-sim_distribution/vectorAdd# grep -i "total_cache_miss_rate" output.txt && grep -i "gpu_ipc" output.txt

L1D_total_cache_miss_rate = 0.5692

L2_total_cache_miss_rate = 0.1111

gpu_ipc = 178.2117

Submitted by: Suhas Shastry Hebbe Sathish